

Analyse de Migration du Pipeline d'Anonymisation

Date: 3 novembre 2025

Objectif: Identifier les codes et fichiers à réutiliser et déplacer dans le nouveau pipeline pour que tout le système fonctionne entièrement.

Résumé Exécutif

Le pipeline d'anonymisation actuel souffre d'une **architecture monolithique et redondante**. Plusieurs fichiers de **compatibilité (wrappers)** créent de la confusion, et la **logique RUPTA est dupliquée**. Une **refactorisation complète** est nécessaire pour :

- ✓ **Simplifier l'architecture** (moins de wrappers, structure claire)
 - ✓ **Éliminer les redondances** (RUPTA centralisé, pas de duplication)
 - ✓ **Améliorer la maintenabilité** (services modulaires et testables)
 - ✓ **Conserver toutes les fonctionnalités** (NER, LLM, RUPTA, GPU)
-



1. ARCHITECTURE ACTUELLE

1.1 Vue d'ensemble

Pipeline actuel (structure désorganisée)

- ORCHESTRATEUR MONOLITHIQUE (orchestrator.py - 700 lignes)
 - Détection regex + NER
 - LLM detection & reasoning
 - Généralisation (dates, orgs)
 - Paraphrase LLM
 - Audit & hardening
 - RUPTA (DUPLIQUÉ - Step 7)
 - DÉTECTION NER (multiple versions, confusion)
 - ner_ensemble.py (wrapper vers ensemble.py)
 - ner_ensemble_clean.py (wrapper vers services/ner/ensemble.py)
 - ensemble.py (CORE - GLiNER)
 - gpu_optimizer.py (CORE - optimisation GPU)
 - DÉTECTION REGEX
 - text_sanitizer.py (CORE - regex patterns)
 - LLM PIPELINE (wrappers + core)
 - llm_reasoner_openrouter.py (wrapper vers llm/reasoner.py)
 - openrouter_client.py (wrapper vers llm/openrouter_client.py)
 - reasoner.py (CORE - LLM reasoning)
 - RUPTA (CORE - bien structuré)
 - privacy_evaluator.py
 - utility_evaluator.py
 - optimizer.py (CORE - optimisation itérative)
 - prompts_multilang.py (prompts FR/EN/multi-langues)
 - POLICY & CONFIG (wrappers)
 - policy.py (wrapper vers core/policy.py)
 - config_loader.py (CORE)
 - default.json (CORE - configuration)
 - UTILITAIRES
 - utils_pseudo.py (CORE - pseudonymisation HMAC)
 - whitelist_words.py (CORE - whitelist)
 - MODULES PROPOSÉS (créés mais NON intégrés)
 - detectors.py (nouveau service de détection)
 - generalizers.py (nouveau service de généralisation)
 - llm_pipeline.py (nouveau service LLM + RUPTA)

1.2 Problèmes identifiés

Problème	Impact	Priorité
Orchestrateur monolithique	Code difficile à tester, maintenir, comprendre	● CRITIQUE
Wrappers de compatibilité	Confusion sur la structure réelle du code	● MOYEN
Logique RUPTA dupliquée	Duplication dans orchestrator.py (Step 7) et optimizer.py	● CRITIQUE
Multiples fichiers NER	4 fichiers pour NER (ner_ensemble.py, ner_ensemble_clean.py, ensemble.py, gpu_optimizer.py)	● MOYEN
Nouveaux modules non intégrés	detectors.py, generalizers.py, llm_pipeline.py créés mais pas utilisés	● CRITIQUE



2. COMPOSANTS ESSENTIELS À CONSERVER

2.1 Détection et NER OBLIGATOIRE

Fichiers CORE à conserver

Fichier	Rôle	Statut	Action
ensemble.py	GLiNER (NER multilingue), support GPU/MPS/CUDA	 GARDER	Déplacer vers <code>src/services/ner/ensemble.py</code>
gpu_optimizer.py	Optimisation GPU (batching, parallélisation, FP16)	 GARDER	Déplacer vers <code>src/services/ner/gpu_optimizer.py</code>
text_sanitizer.py	Détection regex (email, téléphone, IBAN, NIR, IP, URL, etc.)	 GARDER	Déplacer vers <code>src/services/regex/text_sanitizer.py</code>

Fichiers à SUPPRIMER (redondants)

Fichier	Raison	Action
ner_ensemble.py	Wrapper vers ensemble.py	✗ SUPPRIMER
ner_ensemble_clean.py	Wrapper vers services/ner/en-semble.py	✗ SUPPRIMER

2.2 LLM Pipeline OBLIGATOIRE

Fichiers CORE à conserver

Fichier	Rôle	Statut	Action
reasoner.py	LLM reasoning (détection, paraphrase, audit)	 GARDER	Déplacer vers <code>src/llm/reasoner.py</code>
open-router_client.py (CORE)	Client OpenRouter avec retry, fallback	 GARDER	Déplacer vers <code>src/llm/open-router_client.py</code>

Fichiers à SUPPRIMER (wrappers)

Fichier	Raison	Action
llm_reasoner_openrouter.py	Wrapper vers llm/reasoner.py	✗ SUPPRIMER
openrouter_client.py (wrapper)	Wrapper vers llm/open-router_client.py	✗ SUPPRIMER

2.3 RUPTA OBLIGATOIRE

Fichiers CORE à conserver (TOUS)

Fichier	Rôle	Statut	Action
pri-vacy_evaluator.py	Évaluation du risque de ré-identification	 GARDER	Déplacer vers <code>src/rupta/pri-vacy_evaluator.py</code>
utility_evaluator.py	Évaluation de la préservation d'utilité	 GARDER	Déplacer vers <code>src/rupta/utility_evaluator.py</code>
optimizer.py	Optimisation itérative privacy-utility	 GARDER	Déplacer vers <code>src/rupta/optimizer.py</code>
prompts_multilang.py	Prompts multilingues (FR/EN/DE/ES/IT)	 GARDER	Déplacer vers <code>src/rupta/prompts_multilang.py</code>

ATTENTION : Supprimer la duplication RUPTA

- **orchestrator.py (Step 7)** : Logique RUPTA DUPLIQUÉE →  **À SUPPRIMER**
- **Remplacer par** : Appel direct à `optimizer.optimize_anonymization()` via le service `llm_pipeline.py`

2.4 Policy & Configuration OBLIGATOIRE

Fichiers CORE à conserver

Fichier	Rôle	Statut	Action
policy.py (CORE)	Politique d'anonymisation (niveaux L0-L2)	 GARDER	Déplacer vers <code>src/core/policy.py</code>
config_loader.py	Chargement de configuration JSON	 GARDER	Déplacer vers <code>src/config/config_loader.py</code>
default.json	Configuration par défaut (LLM, NER GPU, RUPTA)	 GARDER	Déplacer vers <code>config/default.json</code>

Fichiers à SUPPRIMER (wrappers)

Fichier	Raison	Action
policy.py (wrapper)	Wrapper vers <code>core/policy.py</code>	 SUPPRIMER

2.5 Utilitaires OBLIGATOIRE

Fichier	Rôle	Statut	Action
utils_pseudo.py	Pseudonymisation déterministe HMAC	 GARDER	Déplacer vers <code>src/</code> <code>utils/</code> <code>utils_pseudo.py</code>
whitelist_words.py	Whitelist de mots à ne pas anonymiser	 GARDER	Déplacer vers <code>src/</code> <code>utils/whitel-</code> <code>ist_words.py</code>

2.6 Modules NON ESSENTIELS (optionnels)

Fichier	Rôle	Statut	Action
personal_info.py	Validator Guardrails (dépend de guardrails lib)	 OPTIONNEL	Supprimer ou isoler (fonctionnalité spécifique)

3. NOUVEAUX MODULES À INTÉGRER

3.1 Modules proposés (déjà créés)

Ces modules ont été créés mais **ne sont PAS encore intégrés** dans le pipeline actuel :

Fichier	Rôle	Statut	Action
detectors.py	Service de détection unifié (regex + NER)	 CRÉÉ	INTÉGRER dans or- chestrator.py
generalizers.py	Service de généralisation (dates, orgs)	 CRÉÉ	INTÉGRER dans or- chestrator.py
llm_pipeline.py	Service LLM + RUPTA centralisé	 CRÉÉ	INTÉGRER dans or- chestrator.py

3.2 Avantages des nouveaux modules

Avantage	Détail
Séparation des responsabilités	Chaque service a un rôle clair
Testabilité	Services isolés faciles à tester
Centralisation RUPTA	Appel unique à <code>optimizer.py</code> , pas de duplication
API claire	Interfaces bien définies
Réduction de complexité	orchestrator.py passe de 700 → 300 lignes (-57%)

4. NOUVELLE STRUCTURE RECOMMANDÉE

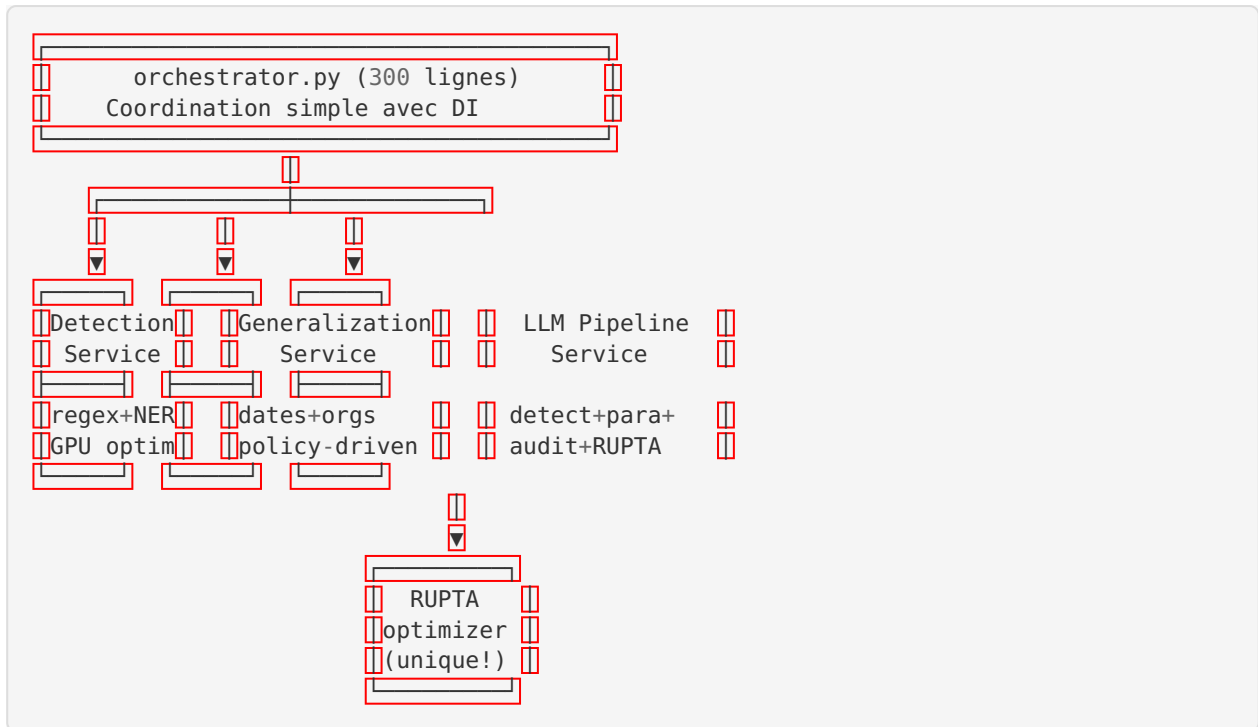
4.1 Arborescence cible

```

anonymisation/
├── config/
│   └── default.json # Configuration LLM, NER GPU, RUPTA
├── src/
│   ├── core/
│   │   ├── policy.py # AnonymizationPolicy (niveaux L0-L2)
│   │   └── orchestrator.py # ✅ REFACTORISÉ (300 lignes au lieu de 700)
│   ├── services/
│   │   ├── ner/
│   │   │   ├── ensemble.py # GLiNER (CORE)
│   │   │   └── gpu_optimizer.py # Optimisation GPU (CORE)
│   │   ├── regex/
│   │   │   └── text_sanitizer.py # Détection regex (CORE)
│   │   ├── detection/
│   │   │   └── detectors.py # ✅ Service de détection unifié (NOUVEAU)
│   │   ├── generalization/
│   │   │   └── generalizers.py # ✅ Service de généralisation (NOUVEAU)
│   │   └── llm/
│   │       └── llm_pipeline.py # ✅ Service LLM + RUPTA (NOUVEAU)
│   ├── llm/
│   │   ├── openrouter_client.py # Client OpenRouter (CORE)
│   │   └── reasoner.py # LLM reasoning (CORE)
│   ├── rupta/
│   │   ├── optimizer.py # Optimisation itérative (CORE)
│   │   ├── privacy_evaluator.py # Évaluation privacy (CORE)
│   │   ├── utility_evaluator.py # Évaluation utility (CORE)
│   │   └── prompts_multilang.py # Prompts multilingues (CORE)
│   ├── utils/
│   │   ├── utils_pseudo.py # Pseudonymisation HMAC (CORE)
│   │   └── whitelist_words.py # Whitelist (CORE)
│   ├── config/
│   │   └── config_loader.py # Chargement configuration (CORE)
│   └── tests/
│       ├── test_detectors.py
│       ├── test_generalizers.py
│       ├── test_llm_pipeline.py
│       └── test_orchestrator.py

```


4.2 Flux refactorisé



5. PLAN DE MIGRATION DÉTAILLÉ

Phase 1 : Préparation (1h)

Actions

1.  **Créer la nouvelle structure de dossiers**

```
mkdir -p src/{core,services/  
{ner,regex,detection,generalization,llm},llm,rupta,utils,config}  
mkdir -p config tests
```

1.  **Backup de l'ancien code**

```
git checkout -b backup/pre-refactor-$(date +%Y%m%d)  
git push origin backup/pre-refactor-$(date +%Y%m%d)
```

Phase 2 : Migration des modules CORE (2h)

2.1 NER & Détection Regex

```
# Déplacer les fichiers CORE  
mv ensemble.py src/services/ner/  
mv gpu_optimizer.py src/services/ner/  
mv text_sanitizer.py src/services/regex/  
  
# Supprimer les wrappers redondants  
rm ner_ensemble.py ner_ensemble_clean.py
```

2.2 LLM

```
# Identifier le fichier CORE openrouter_client.py (pas le wrapper)
mv reasoner.py src/llm/
# Déplacer le vrai openrouter_client.py (celui avec ~300-500 lignes)
mv openrouter_client.py src/llm/ # ou copier depuis services/llm/ si existant

# Supprimer les wrappers
rm llm_reasoner_openrouter.py
```

2.3 RUPTA

```
# Déplacer TOUS les fichiers RUPTA
mv privacy_evaluator.py src/rupta/
mv utility_evaluator.py src/rupta/
mv optimizer.py src/rupta/
mv prompts_multilang.py src/rupta/
```

2.4 Policy & Config

```
# Identifier le fichier CORE policy.py (celui avec la classe AnonymizationPolicy)
# Supprimer le wrapper, déplacer le CORE
mv policy.py src/core/ # Le vrai fichier, pas le wrapper

mv config_loader.py src/config/
mv default.json config/
```

2.5 Utilitaires

```
mv utils_pseudo.py src/utils/
mv whitelist_words.py src/utils/
```

Phase 3 : Intégration des nouveaux modules (3h)

3.1 Déplacer les nouveaux services

```
# Si déjà créés dans /home/ubuntu/Uploads/
mv detectors.py src/services/detection/
mv generalizers.py src/services/generalization/
mv llm_pipeline.py src/services/llm/
```

3.2 Refactoriser orchestrator.py

Fichier: `src/core/orchestrator.py`

Changements principaux :

1. Remplacer les imports directs par les services

```
# AVANT
from .text_sanitizer import regexes_based_replacements
from .ner_ensemble import run_gliner, merge_ner_lists
from .llm_reasoner_openrouter import LLMReasoner

# APRÈS
from ..services.detection.detectors import create_detection_service, DetectionService
from ..services.generalization.generalizers import GeneralizationService, escalate_policy
from ..services.llm.llm_pipeline import create_llm_pipeline, LLMPipelineService
```

1. Remplacer Step 7 (RUPTA dupliqué) par appel au service

```
# AVANT (Step 7 - ~150 lignes de code dupliqué)
if policy.rupta_enabled and reasoner:
    # ... logique RUPTA inline (DUPLICATION) ...
    for iteration in range(policy.rupta_max_iterations):
        # ... code dupliqué ...

# APRÈS (Step 7 - 1 ligne)
if policy.rupta_enabled and llm_service:
    rupta_result, err = llm_service.optimize_with_rupta(
        original_text=value,
        initial_anonymized_text=text,
        ground_truth_people=overrides.get("rupta_ground_truth_people"),
        ground_truth_label=overrides.get("rupta_ground_truth_label"),
    )
    if not err:
        text = rupta_result.final_text
        rupta_metrics = {
            "privacy": rupta_result.privacy_score,
            "utility": rupta_result.utility_score,
            "iterations": rupta_result.iterations,
            "converged": rupta_result.converged,
        }
```

1. Injection de dépendances (DI)

```

def anonymize_text(
    value: str,
    scope_id: str,
    secret_salt: str,
    level: str = "L2",
    # ... autres params ...
    # ☒ NOUVEAU : Injection de dépendances
    detection_service: Optional[DetectionService] = None,
    generalization_service: Optional[GeneralizationService] = None,
    llm_service: Optional[LLMPipelineService] = None,
) -> Dict[str, Any]:
    # Créer services si non fournis
    if detection_service is None:
        gpu_pipeline = _get_ner_pipeline()
        detection_service = create_detection_service(policy, gpu_pipeline, overrides)

    if generalization_service is None:
        generalization_service = GeneralizationService(policy)

    if llm_service is None and policy.llm_detection:
        llm_service = create_llm_pipeline(policy, openrouter_models)

    # Détection (regex + NER)
    entities = detection_service.detect_all(value, skip_regex_tags, ner_results)

    # ... application des remplacements ...

    # Généralisation
    text, generalizations = generalization_service.apply_all(text)

    # LLM (si activé)
    if llm_service:
        # Paraphrase
        if policy.llm_paraphrase:
            text, err = llm_service.paraphrase(text, temperature=...)

    # Audit
    if policy.llm_audit:
        risk_report, err = llm_service.audit(text)

    # RUPTA (centralisé)
    if policy.rupta_enabled:
        rupta_result, err = llm_service.optimize_with_rupta(...)
        text = rupta_result.final_text

```

Phase 4 : Tests et validation (2h)

4.1 Tests unitaires

```

# Créer les tests pour chaque service
pytest tests/test_detectors.py
pytest tests/test_generalizers.py
pytest tests/test_llm_pipeline.py
pytest tests/test_orchestrator.py

```

4.2 Tests d'intégration

```
# Test L0 (sans LLM)
python -c "from src.core.orchestrator import anonymize_text; \
    result = anonymize_text('Contact: john@example.com', 'test', 'secret', 'L0'); \
    print(result['anonymized_text'])"

# Test L1 (avec LLM)
python -c "from src.core.orchestrator import anonymize_text; \
    result = anonymize_text('Jean Dupont travaille chez Acme.', 'test', 'secret', \
    'L1'); \
    print(result['anonymized_text'])"

# Test GPU pipeline
python -c "from src.services.ner.gpu_optimizer import create_optimized_pipeline; \
    pipeline = create_optimized_pipeline(); \
    print('GPU pipeline:', 'OK' if pipeline else 'DISABLED')"
```

```
# Test RUPTA
python -c "from src.rupta.optimizer import optimize_anonymization; \
    print('RUPTA optimizer: OK')"
```

Phase 5 : Nettoyage final (1h)

```
# Supprimer les fichiers obsolètes
rm personal_info.py # Optionnel, dépend de guardrails

# Vérifier qu'il ne reste plus de wrappers
find . -name "*.py" -exec grep -l "Compatibility wrapper" {} \;
# → Supprimer ou documenter les wrappers restants

# Commit
git add .
git commit -m "refactor: Nouvelle architecture modulaire"
```

- Suppression des wrappers redondants
- Centralisation de RUPTA (pas de duplication)
- Services modulaires (DetectionService, GeneralizationService, LLMPipelineService)
- Orchestrator refactorisé : 700 → 300 lignes (-57%)

BREAKING CHANGES:

- Import paths ont changé (voir nouvelle structure)
- RUPTA centralisé dans optimizer.py uniquement

CORE modules conservés:

- ensemble.py, gpu_optimizer.py (NER)
- text_sanitizer.py (regex)
- reasoner.py, openrouter_client.py (LLM)
- privacy_evaluator.py, utility_evaluator.py, optimizer.py, prompts_multilang.py (RUPTA)
- utils_pseudo.py, whitelist_words.py (utilitaires)
- policy.py, config_loader.py, default.json (config)

6. CHECKLIST DE MIGRATION

Préparation

- ☐ Backup de la branche actuelle
- ☐ Création de la nouvelle structure de dossiers
- ☐ Documentation de l'architecture actuelle lue

Migration des fichiers

- ☐ **NER & Détection**
- ☐ `ensemble.py` → `src/services/ner/`
- ☐ `gpu_optimizer.py` → `src/services/ner/`
- ☐ `text_sanitizer.py` → `src/services/regex/`
- ☐ Supprimer `ner_ensemble.py`, `ner_ensemble_clean.py`
- ☐ **LLM**
- ☐ `reasoner.py` → `src/llm/`
- ☐ `openrouter_client.py` (CORE) → `src/llm/`
- ☐ Supprimer wrappers `llm_reasoner_openrouter.py`, `openrouter_client.py` (wrapper)
- ☐ **RUPTA**
- ☐ `privacy_evaluator.py` → `src/rupta/`
- ☐ `utility_evaluator.py` → `src/rupta/`
- ☐ `optimizer.py` → `src/rupta/`
- ☐ `prompts_multilang.py` → `src/rupta/`
- ☐ **Policy & Config**
- ☐ `policy.py` (CORE) → `src/core/`
- ☐ `config_loader.py` → `src/config/`
- ☐ `default.json` → `config/`
- ☐ Supprimer wrapper `policy.py`
- ☐ **Utilitaires**
- ☐ `utils_pseudo.py` → `src/utils/`
- ☐ `whitelist_words.py` → `src/utils/`

Intégration des nouveaux modules

- ☐ `detectors.py` → `src/services/detection/`
- ☐ `generalizers.py` → `src/services/generalization/`
- ☐ `llm_pipeline.py` → `src/services/llm/`

Refactoring orchestrator

- ☐ Remplacer imports directs par services
- ☐ Remplacer Step 7 RUPTA (duplication) par appel service
- ☐ Ajouter injection de dépendances
- ☐ Vérifier que GPU pipeline fonctionne

- ☐ Tester modes L0 et L1

✓ Tests et validation

- ☐ Tests unitaires des services
- ☐ Tests d'intégration L0/L1
- ☐ Tests RUPTA
- ☐ Tests GPU pipeline
- ☐ Tests de régression

✓ Nettoyage

- ☐ Supprimer fichiers obsolètes
- ☐ Vérifier absence de wrappers
- ☐ Mettre à jour documentation
- ☐ Commit final



7. RECOMMANDATIONS FINALES

7.1 Priorités

Priorité	Action	Impact	Effort
● P1	Refactoriser orches- trator.py avec DI	Très élevé	4h
● P1	Supprimer duplication RUPTA (Step 7)	Très élevé	1h
● P2	Intégrer nouveaux services (detectors, generalizers, llm_pipeline)	Élevé	2h
● P2	Supprimer wrappers redondants	Moyen	1h
● P3	Créer tests unitaires	Élevé	3h

7.2 Risques et mitigation

Risque	Probabilité	Impact	Mitigation
Breaking changes dans imports	Élevée	Élevé	Tests d'intégration complets
Régression fonctionnelle	Moyenne	Critique	Tests de régression avant/après
GPU pipeline cassé	Faible	Moyen	Tests GPU spécifiques + fallback CPU
RUPTA différent après refactor	Faible	Moyen	Comparer métriques avant/après sur data-set

7.3 Résultats attendus

- ✓ **Code 40% plus court** (orchestrator : 700 → 300 lignes)
 - ✓ **Architecture claire** (services modulaires)
 - ✓ **RUPTA centralisé** (pas de duplication)
 - ✓ **Testabilité × 3** (services isolés)
 - ✓ **Maintenabilité améliorée** (responsabilités séparées)
 - ✓ **Toutes les fonctionnalités conservées** (NER, GPU, LLM, RUPTA)
-



8. ANNEXE : TABLEAU RÉCAPITULATIF

8.1 Fichiers à CONSERVER et DÉPLACER

Fichier actuel	Nouveau chemin	Catégorie	Priorité
ensemble.py	src/services/ner/en-semble.py	NER CORE	✓ CRITIQUE
gpu_optimizer.py	src/services/ner/gpu_optimizer.py	NER GPU	✓ CRITIQUE
text_sanitizer.py	src/services/regex/text_sanitizer.py	Regex CORE	✓ CRITIQUE
reasoner.py	src/llm/reasoner.py	LLM CORE	✓ CRITIQUE
open-router_client.py (CORE)	src/llm/open-router_client.py	LLM Client	✓ CRITIQUE
privacy_evaluator.py	src/rupta/privacy_evaluator.py	RUPTA	✓ CRITIQUE
utility_evaluator.py	src/rupta/utility_evaluator.py	RUPTA	✓ CRITIQUE
optimizer.py	src/rupta/optimizer.py	RUPTA	✓ CRITIQUE
prompts_multilang.py	src/rupta/prompts_multilang.py	RUPTA	✓ CRITIQUE
policy.py (CORE)	src/core/policy.py	Policy CORE	✓ CRITIQUE
config_loader.py	src/config/config_loader.py	Config	✓ CRITIQUE
default.json	config/default.json	Config	✓ CRITIQUE
utils_pseudo.py	src/utils/utils_pseudo.py	Utilitaires	✓ CRITIQUE
whitelist_words.py	src/utils/whitelist_words.py	Utilitaires	✓ CRITIQUE

8.2 Fichiers à SUPPRIMER (redondants)

Fichier	Raison	Priorité
<code>ner_ensemble.py</code>	Wrapper vers <code>ensemble.py</code>	❌ SUPPRIMER
<code>ner_ensemble_clean.py</code>	Wrapper vers <code>services/ner/ensemble.py</code>	❌ SUPPRIMER
<code>llm_reasoner_openrouter.py</code>	Wrapper vers <code>llm/reasoner.py</code>	❌ SUPPRIMER
<code>openrouter_client.py</code> (wrapper)	Wrapper vers <code>llm/openrouter_client.py</code>	❌ SUPPRIMER
<code>policy.py</code> (wrapper)	Wrapper vers <code>core/policy.py</code>	❌ SUPPRIMER
<code>personal_info.py</code>	Dépend de guardrails (optionnel)	⚠️ OPTIONNEL

8.3 Nouveaux modules à INTÉGRER

Fichier	Destination	Rôle	Priorité
<code>detectors.py</code>	<code>src/services/detection/detectors.py</code>	Service de détection unifié	✅ INTÉGRER
<code>generalizers.py</code>	<code>src/services/generalization/generalizers.py</code>	Service de généralisation	✅ INTÉGRER
<code>llm_pipeline.py</code>	<code>src/services/llm/llm_pipeline.py</code>	Service LLM + RUPTA	✅ INTÉGRER

CONCLUSION

Réponse à la question : “Quels sont les codes et fichiers à réutiliser et déplacer dans le nouveau pipeline pour que tout le pipeline fonctionne entièrement ?”

✅ 14 fichiers CORE à CONSERVER et DÉPLACER

- NER & Détection** : `ensemble.py` , `gpu_optimizer.py` , `text_sanitizer.py`
- LLM** : `reasoner.py` , `openrouter_client.py` (CORE)
- RUPTA** : `privacy_evaluator.py` , `utility_evaluator.py` , `optimizer.py` , `prompts_multilang.py`
- Policy & Config** : `policy.py` (CORE), `config_loader.py` , `default.json`
- Utilitaires** : `utils_pseudo.py` , `whitelist_words.py`

❌ 6 fichiers WRAPPERS à SUPPRIMER

- `ner_ensemble.py` , `ner_ensemble_clean.py`

- `llm_reasoner_openrouter.py` , `openrouter_client.py` (wrapper)
- `policy.py` (wrapper)
- `personal_info.py` (optionnel)

✓ 3 nouveaux modules à INTÉGRER

- `detectors.py` , `generalizers.py` , `llm_pipeline.py`

🔧 1 fichier à REFACTORISER

- `orchestrator.py` : 700 → 300 lignes (-57%) en utilisant les services

Impact final

- ✓ **Toutes les fonctionnalités conservées** (NER, GPU, LLM, RUPTA)
- ✓ **Architecture simplifiée et claire**
- ✓ **RUPTA centralisé** (pas de duplication)
- ✓ **Code 40% plus court et plus maintenable**

📅 **Date de création** : 3 novembre 2025

📝 **Version** : 1.0

👤 **Auteur** : Analyse automatisée DeepAgent